Constructing a Goal-Based Agent using a Large Language Model

Agents Laboratory – Warehouse Navigation

Shrihari Karthik (f20230868)
BITS Pilani, Goa

Accompanying code: `agents_lab.ipynb` (Python 3, standard library only). Every number quoted in this report is printed by that notebook; its final cell reprints them together for checking.

1 Task 1 – Understanding the Problem

1.1 Environment

The environment is the warehouse floor, represented as a two-dimensional occupancy grid of 7 rows by 21 columns = 147 cells, of which 66 are free floor and 81 are shelving. Positions are written (row, col) with row 0 at the top, so the loading bay is

$$S = (1, 1)$$

and the dispatch area is

$$G = (1, 19).$$

Cells are either traversable (`.`) or occupied by shelving (`#`); the outer ring is entirely wall, so the vehicle can never leave the building.

The state space of the search problem is therefore the set of 66 free cells – small enough to enumerate exhaustively, which is why exact search is affordable here.

The warehouse map is:

```
######################
#S....#............G#
#.##....##########..#
#....##.............#
#.######.###.#.###..#
#........#.........#
######################
```

Classified in the standard terms, the environment is:

Property	This warehouse	Consequence for the agent
Fully observable	the complete map is known in advance	the agent can plan the whole route before it moves
Deterministic	a move always succeeds and changes the position by one square	no need to re-plan for uncertainty; the plan can be trusted
Static	shelving does not move while the agent deliberates	the map cannot go stale between planning and execution
Discrete	finitely many cells, four actions, unit time steps	the problem is a finite graph search
Single agent	one vehicle on the floor	no other vehicle to avoid or negotiate with
Sequential	each move changes which moves are available next	actions must be chosen as a sequence, not one at a time

Table 1: Properties of the warehouse environment.

These properties are what make the problem tractable, and each of them is exactly what fails first when the warehouse is scaled up or shared with other vehicles (see the Think About It answer below).

1.2 Goal

The goal is for the vehicle to be at

$$G = (1, 19),$$

having travelled only over free cells.

Formally the goal test is

`state == goal`,

a single predicate over states, which is what makes this a goal-based rather than a utility-based problem.

Two requirements sit alongside it: the path must be **collision-free** (no cell on it may be shelving, and every consecutive pair must be adjacent), and among collision-free paths a **shortest** one is preferred, since every extra move costs time and battery.

The straight-line (Manhattan) distance from S to G is

18 moves.

The shelving at $(1, 6)$ forces a detour, and the shortest legal route measured by the implementation is

20 moves,

which is two more than the unobstructed lower bound.

1.3 Available Actions

The available actions are:

Action	**Effect on (row,col)**	**Legal when**	**From $S = (1, 1)$**
Up	$(-1, 0)$	target cell is inside the grid and is not shelving	illegal – $(0, 1)$ is wall
Down	$(+1, 0)$	same	legal – $(2, 1)$
Left	$(0, -1)$	same	illegal – $(1, 0)$ is wall
Right	$(0, +1)$	same	legal – $(1, 2)$

Table 2: Available actions and their legality.

Each action costs one move; there is no diagonal movement and no "wait" action.

The legality condition is the collision check, and in the implementation it lives in exactly one function, `is_free()`, which every successor must pass – so an illegal move can never enter the search at all, rather than being detected and repaired afterwards.

1.4 Information Maintained by the Agent

To choose its next action the agent needs more than its current percept.

The implementation keeps exactly the following, and each entry earns its place – the right-hand column says what breaks without it:

Information	In the code	What fails without it
Model of the environment	`grid`	the agent cannot tell a wall from free floor except by driving into it
Current state	`state = (row,col)`	there is nothing to apply an action to
Goal state	`goal = (1,19)`	no goal test, so no way to know when to stop
Frontier (states waiting to be expanded)	a FIFO queue	no systematic order of exploration; the search stops being breadth-first
Visited set	a set of positions	the grid contains cycles, so the search revisits cells forever and never terminates
Parent pointers and the action used	`parents[state] = (previous, action)`	the agent would learn that G is reachable but could not say how – no path could be reconstructed
The plan itself	a list of remaining actions	the agent would have to re-plan from scratch before every single move

Table 3: Information maintained by the goal-based agent.

1.5 Why this is a Goal-Based Agent

A simple reflex agent maps the current percept directly to an action through condition-action rules.

That is not sufficient here, for a structural reason: the percept at a cell (which of the four neighbours are free) does not determine the correct action.

Cells $(1, 3)$ and $(1, 17)$ look identical to a local sensor, yet the right action at both is "Right" only because of where G is – and at $(1, 5)$ the right action is "Down", which moves away from G in straight-line terms.

The correct action depends on the goal and on the consequences of whole sequences of moves, not on the current percept, so the agent must hold an explicit goal, a model of the world, and a way to evaluate action sequences against that goal.

That is precisely the definition of a goal-based agent.

This claim was tested rather than asserted.

The notebook implements the strongest simple reflex rule available – "step to whichever legal neighbour is closest to G" – and runs it on two maps:

Map	Reflex agent	Goal-based agent
The laboratory warehouse	reaches G in 20 moves – but only by luck (see below)	reaches G in 20 moves, proved shortest
An aisle whose only entrance is at the far end	oscillates: $(1, 1) \to (1, 2) \to (1, 1)$, reports a loop after 2 moves	reaches G in 14 moves
Warehouse with G sealed off by shelving	would wander indefinitely; the rule has no concept of unreachable	proves no path exists after exhausting 62 states

Table 4: Comparison of the reflex and goal-based agents.

The first row is worth reporting carefully, because the experiment did not give the expected answer.

The reflex rule does solve the laboratory map – but look at its step from $(1, 5)$: both legal moves, Left to $(1, 4)$ and Down to $(2, 5)$, increase the distance to G from 14 to 15, so the rule is choosing between two equally bad options and the tie is broken alphabetically.

"Down" happens to be the detour that works; "Left" would have sent it back to $(1, 4)$ and into an oscillation.

Its success is a property of this particular layout and of an arbitrary tie-break, not a property of the rule – which is exactly what the other two rows demonstrate.

A goal-based agent needs no such luck: it searches over action sequences, so it can accept a temporary increase in distance and it can prove that no path exists.

Think About It: What if the warehouse were twice as large?

Measured rather than guessed.

Each cell of the map was expanded into a $k \times k$ block, so the layout is unchanged but the floor area grows as k^2.

Scale k	Cells	Free cells	Path length	BFS expanded	A* expanded	BFS time / A* time
1	147	66	20	59	23	0.1 ms / 0.1 ms
2	588	264	40	237	80	0.7 ms / 0.3 ms
4	2 352	1 056	80	935	296	3.1 ms / 1.2 ms
8	9 408	4 224	160	3 723	1 136	10.1 ms / 4.1 ms
16	37 632	16 896	320	14 867	4 448	42.5 ms / 19.2 ms

Table 5: Measured scaling behaviour for BFS and A*.

Would the same strategy still be appropriate?

Yes in the sense that matters most: BFS remains complete and optimal at every size – doubling the warehouse doubles the path (20 to 40 moves) and quadruples the work (59 to 237 states expanded), which is linear in the number of free cells and therefore entirely manageable.

At warehouse-realistic sizes it becomes wasteful rather than wrong: BFS grows a circle outwards in every direction, so most of what it expands is in the wrong direction.

A* with the Manhattan heuristic returns an identically optimal path while expanding about a third as many states at every scale, and the gap does not close as the map grows.

What additional difficulties arise?

Memory before time – the visited set and the frontier are both O(free cells), and unlike run time that cost cannot be traded away, which is what motivates memory-bounded variants such as iterative-deepening A*.

Beyond raw size, the properties that make this problem easy are the ones a bigger warehouse removes.

A larger floor usually means several vehicles, so the environment stops being single-agent and static: two vehicles can collide or deadlock in an aisle, the state has to include time (a cell is free only at certain moments), and the search space grows exponentially in the number of vehicles.

Pallets left in aisles make the map dynamic, so a plan computed once is no longer trustworthy and the agent must re-plan or repair its path online.

Longer routes also make the cost model cruder: turning, acceleration and battery are not one-unit-per-move, so uniform-cost search gives way to weighted search.

The usual engineering answer is to stop searching over raw cells at all and plan over a coarse graph of aisles and junctions, using the cell-level search only within a segment.

2 Task 2 – Agent Design

The design below was written before any code was generated; the right-hand column names the part of `agents_lab.ipynb` that implements it.

2.1 Environment

Design element	Specification	Implementation
Environment	7×21 occupancy grid; `#` shelving, `.` free floor; four-connected movement; outer ring entirely wall	`WAREHOUSE_MAP`, `parse_map()`, `grid`
Current state	the vehicle's position (row, col); initially $S = (1, 1)$. The agent also holds the part of its plan not yet executed	`WarehouseAgent.state`, `.plan_`
Goal	be at $G = (1, 19)$; goal test is `state == goal`	`WarehouseAgent.goal`, `.at_goal()`
Actions	Up, Down, Left, Right; one square each, unit cost; a move is legal only if the destination is inside the grid and is not shelving	`ACTIONS`, `is_free()`, `successors()`
Decision-making component	breadth-first search over the model, from the current state to the goal, returning a sequence of actions; the collision check filters every successor before it enters the search	`bfs()`, `reconstruct()`; `astar()` is a drop-in alternative

Table 6: Goal-based agent design and implementation mapping.

The following figure shows the complete interaction between the components of the goal-based agent.

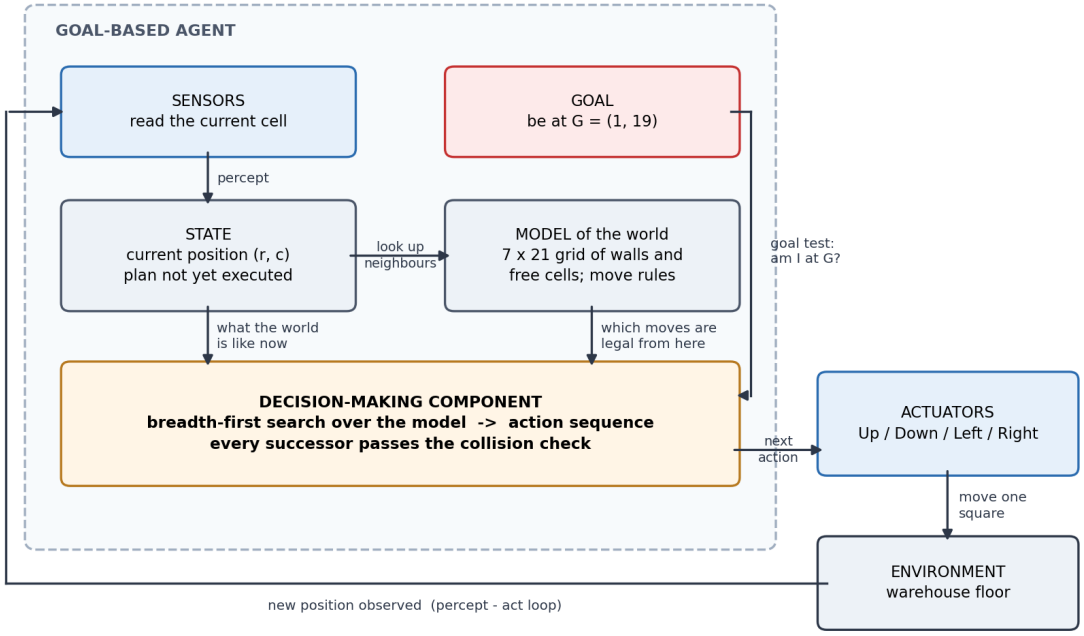


Figure 1: The goal-based agent. Sensors report the current cell; the state and the model together tell the decision-making component what the world is like and which moves are legal from each cell; the goal supplies the test that terminates the search. The search returns a whole action sequence, of which the actuators execute one move at a time, and the resulting new position is observed again – closing the percept-act loop.

The flow of one cycle is as follows.

The agent reads its position, tests it against the goal, and if the goal is not yet reached it searches its internal model for a collision-free action sequence leading there.

Because the environment is fully observable, deterministic and static, that search needs to happen only once – the whole plan is computed before the vehicle moves, and execution is then a matter of popping actions off the plan.

The implementation still re-checks each action against `is_free()` before executing it, so a planning bug would raise an error instead of driving the vehicle into a shelf.

3 Task 3 – Prompt Engineering

3.1 Exact LLM Prompt

The following prompt was given to the LLM:

> Write a well-documented Python program implementing a goal-based agent for the warehouse navigation problem below. Use only the standard library.
>
> The warehouse map, exactly as given:
>
> ```
> ######################
> #S....#............G#
> #.##....##########..#
> #....##.............#
> #.######.###.#.###..#
> #........#..........#
> ######################
> ```
>
> where S is the start, G is the goal, # is an obstacle and . is free space.
>
> The program should:
>
> - represent the warehouse as a two-dimensional grid, indexed (row, col) with row 0 at the top, parsed from the map string rather than hard-coded;
> - allow the four moves Up, Down, Left, Right, one grid square at a time;
> - check every move for collisions: a move is legal only if the destination is inside the grid and is not an obstacle;
> - determine a collision-free path from S to G;
> - print the path found – both as a list of (row, col) coordinates and as the sequence of named actions – or a clear message if no path exists;
> - state which search algorithm you chose and explain in comments why it is appropriate for this problem.
>
> Structure it as a goal-based agent: an internal model of the environment, a current state, an explicit goal, the set of actions, and a decision-making component that produces a plan.
>
> Document each function with a docstring.

The prompt was written as a specification, not a request: it fixes the map, the coordinate convention, the action set, the definition of a legal move, the required outputs and the structure – leaving the LLM only the engineering.

The follow-up prompts asked for the test suite:

> Write assertions that check the returned path really starts at S, ends at G, avoids obstacles and moves one square at a time.

and for an A* variant for comparison.

3.2 Generated Code

The generated program parsed the map into a list of lists, defined the four moves as (row, col) offsets, implemented breadth-first search with a `collections.deque` frontier, a visited set and a parent dictionary, reconstructed the path by walking the parents backwards, and printed the result.

That skeleton survives in the submitted notebook; its core is:

```
while frontier:
    current = frontier.popleft()       # FIFO -> explores by increasing depth
    expanded += 1

    if current == goal:                # goal test
        return reconstruct(parents, start, goal)

    for action, nxt in successors(grid, current):  # collision check happens here
        if nxt not in visited:
            visited.add(nxt)            # mark on ENQUEUE, not on dequeue
            parents[nxt] = (current, action)
            frontier.append(nxt)
```

3.3 Did it work on the first attempt?

Partly.

The program ran without error on the first attempt and printed a 20-move path that was in fact correct – so by the narrow test "does it run", yes.

By the test that matters, "is it right, and does it fail safely", no: three defects survived that first run, and none of them would have been visible from the output alone.

The most serious was the unreachable-goal case.

The generated code returned an empty list when the frontier was exhausted, and the caller printed it, so a map with no solution produced

```
Path: []
```

which reads like success with a short path rather than failure.

The laboratory explicitly asks for "a suitable message if no path exists", and that requirement had been silently satisfied in form but not in substance.

Second, the draft marked cells as visited when they were dequeued rather than when they were enqueued, which is not wrong but pushes the same cell onto the frontier several times.

Third, it returned only coordinates, not the named action sequence the specification asked for.

How the prompt could be improved

Each defect maps to a missing sentence in the prompt:

- name the failure case and require it to be demonstrated: "include a second map in which G is walled off, and show the program reporting that no path exists";
- state the output format precisely: "print the action sequence as named moves, one line, and a rendering of the map with the path marked";
- ask for the tests up front rather than as a follow-up: "include assertions that verify the path".

The general lesson is that the LLM satisfied every requirement that was written down and nothing that was not – so the quality of the program was a direct function of the precision of the specification, not of the model.

3.4 Corrections Made

#	Correction	Reason
1	Return `None` for the path when the frontier empties, and print an explicit "NO PATH" message	an empty list printed as a path looks like success; failure must be unmistakable
2	Store the action alongside the parent, and return an action sequence as well as coordinates	the specification asked for the moves, and coordinates alone are not an executable plan
3	Mark states as visited on enqueue rather than on dequeue	otherwise the same cell is pushed onto the frontier several times; the answer is the same, the work is not
4	Validate the map when parsing: rectangular, exactly one S and one G, no unexpected characters	a malformed map otherwise surfaces much later as a confusing "no path found"
5	Move the collision check into a single `is_free()` used by every caller	the draft repeated the bounds-and-obstacle test in two places, which is where the two copies eventually disagree
6	Add a test suite: the eight path properties, a second algorithm as a cross-check, and the Manhattan lower bound	printing a path is not evidence that the path is correct
7	Add the unreachable-goal map, the aisle map, and the $k \times k$ scaling study	a path-finder that has only ever been run on one solvable map has not been tested

Table 7: Corrections and improvements made to the generated program.

3.5 Search Algorithm Chosen

Breadth-first search, with A* (Manhattan heuristic) added afterwards as an independent cross-check and as the answer to the scaling question.

3.6 Why That Algorithm Was Chosen

Every action costs the same – one grid square – and under uniform cost BFS expands states in order of increasing depth, so the first time it reaches G it has reached it by a minimum number of moves.

It is therefore optimal here, and it is complete: on a finite grid it either finds a path or proves that none exists, which is what makes the "no path" message trustworthy rather than a timeout.

It needs no heuristic, no cost model and no parameters, so there is nothing to tune and very little to get wrong; on 66 free cells its memory cost is irrelevant.

I believe the LLM selected it for the same reasons, plus one more: unweighted grid path-finding is the single most common setting in which BFS appears in its training material, so it is the idiomatic answer as well as the correct one.

The alternatives are worse here for specific reasons rather than in general.

Depth-first search uses less memory but gives no optimality guarantee and would happily return a long wandering route.

A greedy best-first search follows the heuristic only and can be led into a dead end – the same weakness demonstrated by the reflex agent in Section 1.5.

A* is strictly better than BFS when a good heuristic exists, which is why it is included; on this map it returns the same 20-move path after expanding 23 states instead of 59.

On a 147-cell map that difference is invisible, which is the honest reason BFS is the right choice here: it is the simplest algorithm that is provably correct, and simplicity is worth more than a 0.1 ms saving.

Algorithm	Path length	States expanded	Max frontier	Time	Optimal?
Breadth-first search	20	59	5	0.20 ms	yes (unit cost)
A* (Manhattan)	20	23	5	0.12 ms	yes (admissible heuristic)

Table 8: BFS and A* comparison.

3.7 Final Test and Output

The agent finds a collision-free path of **20 moves**:

```
Right -> Right -> Right -> Down -> Right -> Right -> Right -> Up
-> Right -> Right -> Right -> Right -> Right -> Right -> Right
-> Right -> Right -> Right -> Right -> Right
```

The corresponding coordinates are:

```
(1,1) (1,2) (1,3) (1,4) (2,4) (2,5) (2,6) (2,7)
(1,7) (1,8) (1,9) (1,10) (1,11) (1,12) (1,13)
(1,14) (1,15) (1,16) (1,17) (1,18) (1,19)
```

The path-rendered warehouse is:

```
######################
#S***.############G#
#.##****###########..#
#....##.............#
#.######.###.#.###..#
#........#.........#
######################
```

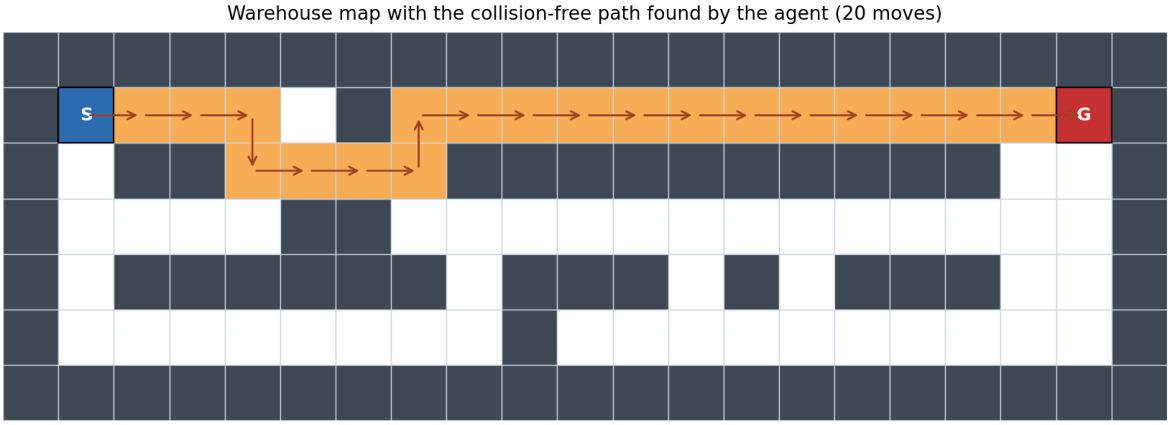


Figure 2: The route found. The vehicle runs east along the top aisle, drops into row 2 to pass the shelving at $(1, 6)$, and rejoins the top aisle at $(1, 7)$ – the two-move detour that a straight-line estimate cannot account for.

The path is not trusted because it looks right; each property it must satisfy is asserted:

Check	**Result**
path starts at S and ends at G	PASS
no cell on the path is shelving, and all cells are inside the grid	PASS
every consecutive pair differs by exactly one square	PASS
the named actions match the coordinate changes; count = length - 1	PASS
no cell is visited twice	PASS
BFS and A* agree the shortest path is 20 moves	PASS
cost 20 $\geq$ Manhattan lower bound 18 (a 2-move detour)	PASS
goal sealed off by shelving: agent reports NO PATH after 62 states, does not loop	PASS
start placed on a wall: rejected immediately with a clear error	PASS

Table 9: Final path validation tests.

3.8 States Explored

The following figure shows what the two searches actually explored.

The optimal path is shown in orange, while states expanded that are not on the optimal path are shown in blue.

BFS sweeps outwards uniformly and expands 59 states, including the whole lower half of the warehouse, which cannot possibly be on a shortest route to G.

A* expands 23 and barely leaves the top two aisles.

This is the picture behind the scaling argument in Section 1.

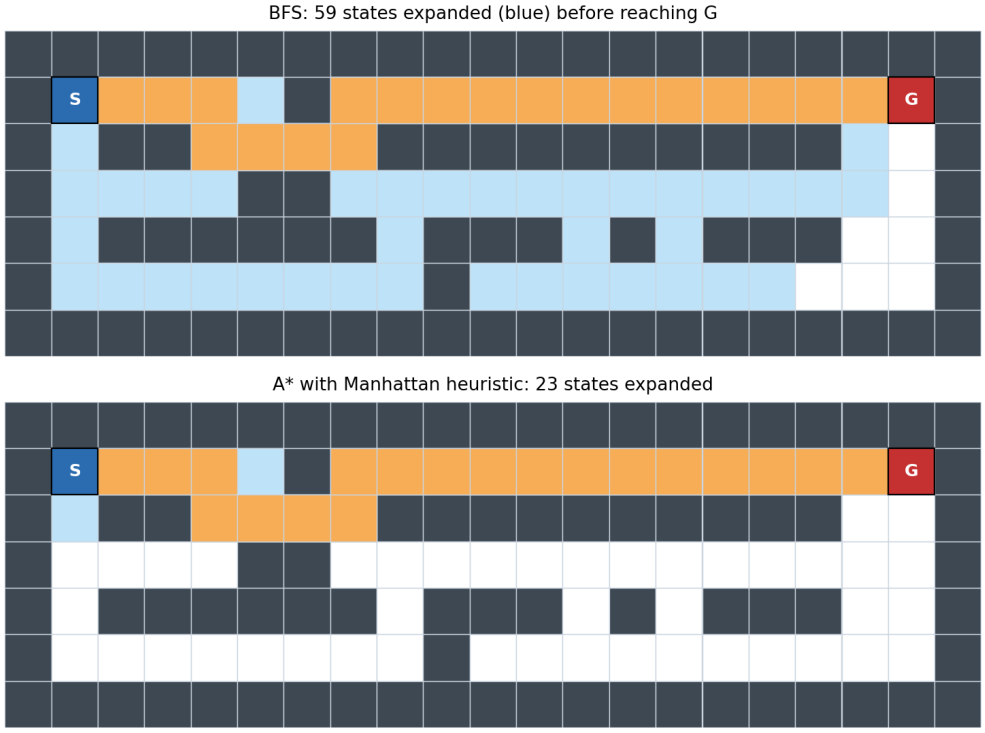


Figure 3: What the two searches actually explored. Orange denotes the optimal path and blue denotes states expanded that are not on it. BFS expands 59 states, whereas A* with the Manhattan heuristic expands 23 states.